The Lambda Calculus — a short explainer

The heart of any functional programming language are the three essential forms of the *lambda calculus*: lambda abstraction (defining a function), application (invoking a function), and variable reference. In fact, the rest of Scheme (or any other programming language) can be compiled down into the language consisting of just these three forms: defining unary (single input) functions that bind a variable when invoked, invoking functions on a single argument expression, and referencing a variable. Compiling down to the pure lambda calculus is called *Church compiling/encoding*, after the creator of the lambda calculus, Alonzo Church.

While there are many different lambda calculi—systems for calculating (calculi) using functions (lambdas)—*the lambda calculus* generally refers to a specific classic system: the untyped, three-form lambda calculus with unary functions. We can define a grammar for its expressions/terms like so:

$$
\begin{aligned}
e, t \in \mathsf{E} ::= {} & (\lambda\ (x)\ e_b) && \lambda x.e_b \\
\mid {} & (e_f\ e_a) && e_f\ e_b \\
\mid {} & x && x \\
x, y \in \mathsf{Var} = {} & \langle \text{program identifiers} \rangle
\end{aligned}
$$

It's also common to see a notation without (required) parentheses and with a dot before the body of lambdas (alternative shown on right)—λ is highest precedence.

Lambda calculus is found within many programming languages; it is the heart of functional programming languages such as Racket, Haskell, OCaml, but is also found within multi-paradigm languages permitting first-class functions such as Python, Ruby, Javascript, and even Java, and C++. First-class functions are now in most modern languages, e.g.:

Python: `id = lambda x: x`

Java: `Function<Object, Object> id = x -> x;`

Javascript: `const id = x => x;`

With an application form, we may apply an identity function on a value, which will yield that value unchanged. Let's take the identity function $(\lambda\ (x)\ x)$ and apply it to the number 5: $((\lambda\ (x)\ x)\ 5)$. This expression may be textually simplified using an evaluation-step relation ($\rightarrow$): $((\lambda\ (x)\ x)\ 5) \rightarrow 5$. In the pure lambda calculus, we do not have the ability to represent a constant like 5, but could still apply the identity function on itself: $((\lambda\ (x)\ x)\ (\lambda\ (y)\ y)) \rightarrow (\lambda\ (y)\ y)$.

1 REDUCING EXPRESSIONS

A *reducible expression*, or *redex*, is an expression that can be simplified by applying a reduction rule. The primary reduction rule for the λ calculus is β reduction, which simulates function application. We define β reduction for an application expression with a lambda, ready to apply, in function position and any expression in argument position:

$$
\overbrace{((\lambda\ (x)\ e_{\mathsf{body}})\ e_{\mathsf{arg}})}^{\text{redex}} \rightarrow_\beta \overbrace{e_{\mathsf{body}}[x \mapsto e_{\mathsf{arg}}]}^{\text{capture-avoiding substitution}}
$$

Given a lambda applied on an argument expression, we can textually simulate function application by substituting every occurrence of the formal parameter in the body of the function with the argument. The notation $e_b[x \mapsto e_a]$ refers to e_b, the body of the function, except with each reference to x swapped with the argument expression e_a. This is called *capture-avoiding substitution* (defined below), which is simply a substitution operation on terms in the λ calculus that respects the rules of scope and binding for the language; for example, substitution must respect shadowing variables:

$$
\begin{aligned}
&((\lambda\ (\mathsf{x})\ (\mathsf{x}\ (\lambda\ (\mathsf{x})\ \mathsf{x})))\ (\lambda\ (\mathsf{y})\ \mathsf{y})) \\
&\quad \rightarrow_\beta (\mathsf{x}\ (\lambda\ (\mathsf{x})\ \mathsf{x}))[x \mapsto (\lambda\ (\mathsf{y})\ \mathsf{y})] \\
&\quad = ((\lambda\ (\mathsf{y})\ \mathsf{y})\ (\lambda\ (\mathsf{x})\ \mathsf{x}))
\end{aligned}
$$

An outer reference to x is replaced with the argument (i.e., $(\lambda\ (\mathsf{y})\ \mathsf{y})$) but an inner reference to x is not. This is because *static scope* dictates that the x in $(\lambda\ (\mathsf{x})\ \mathsf{x})$ always refers to the closest enclosing declaration, making this always the identity function, in all contexts. (None of the variables in $(\lambda\ (\mathsf{x})\ \mathsf{x})$ are free—none are defined outside the function.)

An occurrence of a variable in an expression may be either free or bound. A variable is called *free* in an expression if it is used within that expression but not defined above its use within that expression. λ is a binding operator as it binds a variable within a particular scope (its body). In the identity function, $(\lambda\ (x)\ x)$, the variable x in the body is bound by the λ. However, given the function $(\lambda\ (b)\ a)$, we may conclude the variable a is a free variable in our function, meaning it must be defined in the context surrounding this lambda in order to have a meaning. In the function $(\lambda\ (a)\ (\lambda\ (b)\ a))$, however, a is not free as it is defined by the outer lambda. With respect to the inner lambda, a is free because its binding lambda must be located in the *context* of this expression, surrounding it. A variable alone is never called free, but always relative to a sub-expression (scope).

We may formally define free variables like so:

$$FV(x) \triangleq \{x\}$$
$$FV((e_f\ \ e_a)) \triangleq FV(e_f) \cup FV(e_a)$$
$$FV((\lambda\ (x)\ \ e_b)) \triangleq FV(e_b) - \{x\}$$

To aid in understanding, consider the following examples with their free variables:

`((λ (x) x) y)`	$FV = \{\mathsf{y}\}$
`((λ (x) (x x)) (λ (x) (x x)))`	$FV = \{\}$
`((λ (x) (z y)) x)`	$FV = \{\mathsf{x}, \mathsf{y}, \mathsf{z}\}$

Open λ-calculus terms contain free variables, while *closed* terms have no free variables. Closed lambdas are known as *combinators* and are self-contained functions with a stable interpretation across all contexts. Consider some examples:

`(λ (x) (x x))`	*closed*
`((λ (x) x) y)`	*open*
`(λ (x) ((λ (y) y) ((λ (z) (z z)) d)))`	*open*

As we perform substitution, we must take care to avoid inadvertent capture of any otherwise-bound variables. Let us take the abstract example $((\lambda\ (x)\ e_{\mathsf{body}})\ e_{\mathsf{arg}})$. At a β reduction, we would replace any instances of x within e_{body} with e_{arg}. The two expressions involved within the substitution may use the same variable name at distinct binding sites (lambdas). If so, this could lead to us changing the meaning of the resulting expression if we are not careful. To illustrate this, consider the naïve (non-capture-avoiding) substitution:

$$\begin{aligned}&((\lambda\ (x)\ ((\lambda\ (y)\ x)\ z))\ y)\\ &\rightarrow_\beta ((\lambda\ (y)\ y)\ z)\\ &\rightarrow_\beta \mathsf{z}\end{aligned}$$

We've directly substituted `y` for `x` in the expression `((λ (y) x) y)` and that's how we ended up with `(λ (y) y)`. This changes the meaning of the inner lambda (to match the identity function), because the y we originally wanted to substitute for was a free variable in the overall expression, and not the same y that is bound by the inner lambda. In our resulting expression, the y we substituted became incorrectly captured by the inner λ. Our aim is to avoid this through *capture-avoiding substitution*. This works by preventing such substitutions that would lead to a reference being captured by the wrong binding lambda in cases with shadowing.

We can also simply rename variables to eliminate shadowing, using *α-conversion*. Equivalent expressions that only differ in the names of variables are α-equivalent. *α-conversion* does not alter the structure of the lambda like a β reduction would. E.g., $(\lambda\ (\mathsf{x})\ \mathsf{x}) \equiv_\alpha (\lambda\ (\mathsf{y})\ \mathsf{y})$. In this example, both are equally the identity function. We can define *α equivalence* (or directionally, *α conversion*) using substitution, just as with β, but also must use capture-avoiding substitution:

$$(\lambda\ (x)\ \ e_b) \equiv_\alpha (\lambda\ (y)\ \ e_b[x \mapsto y])$$
$$\text{where } y \notin FV(e_b)$$

Renaming variables can enable a correct substitution that would otherwise be *incorrect* using naïve substitution and *undefined* using capture-avoiding substitution. Let's take a look at the same example but with a preparatory renaming before substitution:

$$\begin{aligned}&((\lambda\ (\mathsf{x})\ ((\lambda\ (\mathsf{y})\ \mathsf{x})\ \mathsf{z}))\ \mathsf{y})\\ &\equiv_\alpha ((\lambda\ (x)\ ((\lambda\ (\mathsf{a})\ \mathsf{x})\ \mathsf{z}))\ \mathsf{y})\\ &\rightarrow_\beta ((\lambda\ (\mathsf{a})\ \mathsf{y})\ \mathsf{z})\\ &\rightarrow_\beta \mathsf{y}\end{aligned}$$

We notice by first α renaming our variables we preserve the meaning of our lambda expression while only changing the names of variables. Now we may formally define *capture-avoiding substitution* so that we stop substitution at a shadowing binding, and forbid substitution where we'd substitute a free variable under a shadowing binding site:

$$y[x \mapsto e_{\mathsf{x}}] \triangleq e_{\mathsf{x}}, \text{where}$$
$$x = y$$
$$y[x \mapsto e_{\mathsf{x}}] \triangleq y, \text{where}$$
$$x \neq y$$
$$(\lambda\ (y)\ \ e_{\mathsf{body}})\ [x \mapsto e_{\mathsf{x}}] \triangleq (\lambda\ (y)\ \ e_{\mathsf{body}}), \text{where}$$
$$x = y$$
$$(\lambda\ (y)\ \ e_{\mathsf{body}})\ [x \mapsto e_x] \triangleq (\lambda\ (y)\ \ e_{\mathsf{body}}[x \mapsto e_x]), \text{ where}$$
$$x \neq y$$
$$y \notin FV(e_{\mathsf{x}})$$
$$(e_f\ \ e_a)\ [x \mapsto e_x] \triangleq (e_f[x \mapsto e_x]\ \ e_a[x \mapsto e_x])$$

We've seen the two principal notions of intensional evaluation: β reduction and α equivalence. Now we'll present a third and final notion of extensional equivalence: η reduction/expansion. While we may avoid η reduction during evaluation (which may use only β), it assists us in establishing meaningful equalities among λ-calculus terms. η equivalence is formally defined like so:

$$(\lambda\ (x)\ \ (\mathsf{e}_f\ \ x)) \equiv_\eta \mathsf{e}_f$$
$$\text{where } x \notin FV(\mathsf{e}_f)$$

In essence, η reduction says any lambda that simply defers to its function-position expression (e_f) may be reduced to that expression. In this way, it anticipates a β reduction that may or may not be performed later and eliminates the need for it, preemptively. η expansion says that every sub-expression

which reduces to a value reduces to a unary lambda equivalent to applying the sub-expression on a fresh formal parameter. η expansion lets us reify an arbitrary expression as the value it denotes by making that lambda explicit. If the expression would non-terminate, this new explicit lambda non-terminates when invoked, and if it yields a value, this new lambda will be equivalent to it. Any expression may be η expanded, but only expressions with the particular shape above may be η reduced.

To summarize, the concepts of free variables, α-conversion, β-reduction, η-reduction and capture-avoiding substitution are fundamental to the inner workings of lambda calculus and functional programming. We've seen how we may apply these reductions non-deterministically, essentially using a combined reduction relation ($\rightarrow$):

$$(\rightarrow) \triangleq (\equiv_\alpha) \cup (\rightarrow_\beta) \cup (\equiv_\eta)$$

We can also define the reflexive, transitive closure of this step relation ($\rightarrow^*$), in order to take multiple steps at once. You can read the proposition $e \rightarrow e'$ as "e steps to e prime" or "e reduces to e prime, in one step", while $e \rightarrow^* e'$ means "e reduces to e prime" (in some number of steps—which may be zero, meaning they are already equal).

By applying these notions of equivalence or reduction to a term, we may eventually reach a *normal form*: a term that cannot be further reduced and has no internal redexes. However, not all terms in the lambda calculus can be reduced to a normal form. The smallest such term, known as Ω, is `((λ (u) (u u)) (λ (u) (u u)))`. Just `(λ (u) (u u))` is known as the U combinator and is the abstraction of self-application. Ω is just `(U U)`, the U combinator applied to itself, which reduces to itself in a single beta reduction.

2 EVALUATION STRATEGIES

We have now seen the principal notions of reduction and equivalence for the λ calculus, applied non-deterministically to redexes within terms. These rules allow us to rewrite expressions to another equivalent expression, which may be simpler or closer to a normal form. α conversion allows us to rename variables, β-reduction simulates function application (i.e., invocation, instantiation), and η expansion or reduction characterizes a notion of extensional equivalence: that any term is equivalent to that term wrapped in a lambda that applies it, in all possible contexts. We can also choose a specific evaluation strategy and enforce deterministic evaluation.

We'll define four primary evaluation strategies: applicative-order evaluation (AOE), normal-order evaluation (NOE), call by value (CBV), and call by name (CBN). In *applicative-order evaluation*, a function's argument is reduced before applying the function itself (i.e., the *rightmost* β redex is always next) while in *normal-order evaluation* we apply the function first without reducing its argument (i.e., the *leftmost* β redex is always next). NOE's delayed evaluation for arguments can also be viewed as a form of *lazy* evaluation, contrasted with the *eager* evaluation of AOE.

Reduction using AOE:

```
   ((λ (y) y) ((λ (z) z) w))
→β ((λ (y) y) w)
→β w
```

Reduction using NOE:

```
   ((λ (y) y) ((λ (z) z) w))
→β ((λ (z) z) w)
→β w
```

Call-by-value (CBV) order is exactly the same as applicative order except it does not reduce under a lambda. Call-by-name (CBN) evaluation is exactly the same as normal-order evaluation except it does not reduce under a lambda. So CBV and CBN are the same as AOE and NOE, respectively, except that we skip over any redexes that are within a (not-yet-applied) lambda.

Reduction using CBV:

```
   ((λ (x) (λ (y) ((λ (z) z) y)))
    ((λ (a) a) (λ (b) b)))
→β ((λ (x) (λ (y) ((λ (z) z) y)))
    (λ (b) b))
→β (λ (y) ((λ (z) z) y))
```

Reduction using CBN:

```
   ((λ (x) (λ (y) ((λ (z) z) y)))
    ((λ (a) a) (λ (b) b)))
→β (λ (y) ((λ (z) z) y))
```

Note that CBV reduces arguments before applying functions on them, similar to AOE, while CBN applies functions on argument expressions before reducing them, similar to NOE. Neither CBV or CBN, however, will evaluate under a lambda until that lambda is applied—exposing its body.

To see a summary example, let's reduce the following lambda expression using all four reduction strategies:

```
((λ (x) (λ (y) (x y))) ((λ (a) a) (λ (w) w)))
```

Reduction using CBV:

```
→β ((λ (x) (λ (y) (x y))) (λ (w) w))
→β (λ (y) ((λ (w) w) y))
```

Reduction using CBN:

```
→β (λ (y) (((λ (a) a) (λ (w) w)) y))
```

Reduction using AOE:

$\rightarrow_\beta$ ((λ (x) (λ (y) (x y))) (λ (w) w))
$\rightarrow_\beta$ (λ (y) ((λ (w) w) y))
$\rightarrow_\beta$ (λ (y) y)

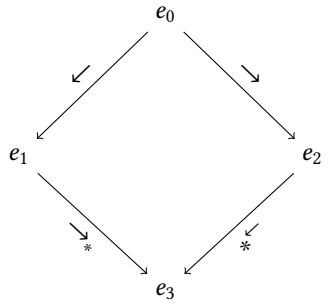
Reduction using NOE:

$\rightarrow_\beta$ (λ (y) (((λ (a) a) (λ (w) w)) y))
$\rightarrow_\beta$ (λ (y) ((λ (w) w) y))
$\rightarrow_\beta$ (λ (y) y)

Notice how AOE mirrors CBV and NOE mirrors CBN until our expression would require reducing under a lambda/value. In this case, both AOE and NOE continue reducing under the lambda expression while CBV and CBN halt evaluation. A table can be made to visually depict the similarities and differences among the four different evaluation strategies:

	incl. under λs	not under λs
Reduce arguments first (rightmost redex)	AOE	CBV
Apply function first (leftmost redex)	NOE	CBN

Notice that in our last example, all orders of evaluation reach equivalent values, though some are not yet in a normal form. That this is always the case is known as the *confluence* or *diamond* property of the lambda calculus and is formalized by the *Church-Rosser Theorem*. This property states that if we start with an expression e_0 and by applying different reductions to it we can step to two different intermediate expressions, e_1, and e_2, then eventually these diverging paths can always reduce further to a common term e_3.



However, n.b. that this does not guarantee that evaluation will always terminate with a value. Let's consider an example where CBN/NOE will terminate but CBV/AOE does not.

Reduction using CBN/NOE:

((λ (a) (λ (b) b)) Ω)
$\rightarrow_\beta$ (λ (b) b)

Using call by name or normal order we apply functions before reducing their arguments, so the Ω subterm is never reduced, avoiding non-termination. If we used call by value or applicative order, the Ω term would be reduced ad infinitum causing evaluation to continue without reaching a value or normal form.

2.1 Evaluation contexts

We can formalize a grammar for evaluation contexts ($\mathcal{E}$) to clearly specify the order in which reducible sub-expressions are simplified. Formally, an evaluation context is an expression with a "hole" (□) located in the position of the current redex. Using a grammar to constrain these contexts, we can enforce different evaluation strategies; for example, CBV:

$\mathcal{E}$ ::= □	[a hole—where the redex goes]
| (e $\mathcal{E}$)	[arg. position may be evaluated first]
| ($\mathcal{E}$ v)	[func. position may be evaluated next]
r ::= (v v)	[a β redex is a λ applied on a λ]
v ::= (λ (x) e)	[values are λs]

Note how this grammar enforces CBV evaluation order by ensuring that redexes are values applied on values, rightmost evaluation, and that a context may not contain a lambda. How might CBN, AOE, and NOE be formalized in this way?

When evaluating a term using a context and redex, we first (deterministically) split the term, t into its context and redex, $t = \mathcal{E}[r]$, β reduce the redex, $r \rightarrow_\beta e$, and then substitute the simplified redex (e) back into its context before taking another step of evaluation: $t' = \mathcal{E}[e]$. Then, if the subsequent term, t', can be split into another context and redex, evaluation continues, and if not, we can terminate with a final value. For example, consider the expression:

$$t_0 = ((\lambda\ (a)\ a)\ ((\lambda\ (b)\ b)\ (\lambda\ (c)\ c)))$$

To take one step, we split t_0 into context and redex:

$$t_0 = \mathcal{E}_0[r_0]$$
$$\mathcal{E}_0 = ((\lambda\ (a)\ a)\ \square)$$
$$r_0 = ((\lambda\ (b)\ b)\ (\lambda\ (c)\ c))$$

After applying β-reduction on r_0, we get e_0:

$$r_0 \rightarrow_\beta \underbrace{(\lambda\ (c)\ c)}_{e_0}$$

This is then plugged back into the context:

$$t_1 = \mathcal{E}_0[e_0] = ((\lambda\ (a)\ a)\ (\lambda\ (c)\ c))$$

We can then take a final step of evaluation. Now our context will be empty and the entire term is a redex:

$$t_1 = \mathcal{E}_1[r_1]$$
$$\mathcal{E}_1 = \square$$
$$r_1 = ((\lambda\ (a)\ a)\ (\lambda\ (c)\ c))$$
$$r_1 \rightarrow_\beta \underbrace{(\lambda\ (c)\ c)}_{e_1}$$
$$t_2 = \mathcal{E}_1[e_1] = (\lambda\ (c)\ c)$$

Finally, since this term (t_2) is a value and cannot be split into context and redex, evaluation terminates.

3 TURING EQUIVALENCE

If a language is Turing equivalent (i.e., of the same expressive power as a Turing machine) then we would expect it to be able to do an unbounded amount of work and therefore non-terminate. As we have seen before, we can express the Ω term in the pure λ calculus, which exemplifies non-termination. But this alone is not too convincing. The question remains: how can we express arbitrary Scheme programs in the pure λ calculus, using only unary function definition, unary function application, and variable reference?

The answer is called Church compiling or Church encoding, and represents either a way of using the λ calculus as a practical programming language or a way of systematically compiling a richer language (like Scheme) down to the λ calculus. There are various alternative approaches to Church encoding basic values like numbers, lists, or Booleans, but we will focus here on a particular, traditional set of encodings.

First, we may encode k-ary functions as unary higher-order functions by taking each parameter one at a time—a technique known as *Currying*, named for logician Haskell Curry who popularized the technique. A binary function thus becomes two nested unary functions:

$$(\lambda\ (x\ y)\ e) \equiv (\lambda\ (x)\ (\lambda\ (y)\ e))$$

We must transform k-ary application into a Curried form as well. For example:

$$(f\ x\ y\ z) \equiv (((f\ x)\ y)\ z)$$

Thunks, or lambdas with no parameters, can be encoded by taking a dummy parameter that goes unused. Similarly, application of a thunk can be converted to provide a arbitrary value as its argument. What about true and false?

$$\#t \equiv (\lambda\ (x)\ (\lambda\ (y)\ x))$$
$$\#f \equiv (\lambda\ (x)\ (\lambda\ (y)\ y))$$

We encode Booleans as selecting functions that take two arguments and select one. The function encoding $\#t$ returns the first argument provided and ignores the second whereas $\#f$ returns the second argument, discarding the first.

We can encode conditional `if` forms as application of the guard expression on the then-and-else subexpressions.

$$(\texttt{if}\ g\ t\ f) \equiv ((g\ t)\ f)$$

In the plain λ calculus, without a fixed evaluation order, or using CBN or Normal evaluation, this encoding will work fine because argument evaluation is delayed/lazy. In CBV evaluation however, we have just changed the termination behavior of a program like `(if #f Ω #f)`, because arguments will be evaluated before selection instead of being delayed. To fix this for CBV, we can η expand the two branches, or explicitly wrap them in thunks and then apply the selected thunk after returning or inside the Boolean (encoded as a function). E.g.,

```
(if g t f) ≡ (((g (λ (_) t))
                (λ (_) f))
               (λ (x) x))
```

We can encode a natural number n as a function that accepts another function f and returns a function that performs f, iterated n times. For example:

$$0 \equiv (\lambda\ (f)\ (\lambda\ (x)\ x))$$
$$1 \equiv (\lambda\ (f)\ (\lambda\ (x)\ (f\ x)))$$
$$2 \equiv (\lambda\ (f)\ (\lambda\ (x)\ (f\ (f\ x))))$$

The encoding for a binary addition function takes two Church encoded numbers, `n` and `m`, and returns a number, i.e., some `(λ (f) (λ (x) ...))`, which encodes the number $n + m$ by using `n` and then `m` to apply `f` n times followed by m times:

```
+ ≡ (λ (n) (λ (m)
             (λ (f) (λ (x)
                      ((m f) ((n f) x))))))
```

The encoding for a binary multiplication function is quite similar, except it exploits the equality $(f^m)^n = f^{(m*n)}$ to accomplish multiplication:

```
* ≡ (λ (n) (λ (m)
             (λ (f) (λ (x)
                      ((n (m f)) x)))))
```

To encode linked lists in Scheme, we need an encoding for cons cells and for null that distinguishes between these two cases and that can destructure a cons cell and give us back its car and cdr values. Null and cons can be encoded, in a similar manner to Booleans, as functions that take two callback functions, invoking the first when it encodes a cons call and the second when it encodes null. The callback for a cons cell takes two values (the contents of the cons cell) while the callback for null is an encoding of a thunk and takes no meaningful parameters.

```
`() ≡ (λ (when-cons)
         (λ (when-null)
           (when-null (λ (x) x))))
```

```
cons ≡ (λ (a) (λ (b)
                (λ (when-cons)
                  (λ (when-null)
                    ((when-cons a) b)))))
```

Thus we can write a definition for `car` like so:

```
car ≡ (λ (p) ((p (λ (a) (λ (b) a)))
              (λ () Ω)))
```

Note that `(car '())` is an error, which we simply represent here as nontermination. Based on the encodings above, how might you write other primitive utilities like `null?` and `cdr`? Finally, we need a way to implement recursion as the plain λ calculus has no recursive binding forms like `define` or `letrec` in Scheme, which we could otherwise use to define a function in terms of itself. Instead, the only binding form we have is λ. A standard approach is to define a fixpoint combinator, which can be used to compute a fixed point for a function, so that to define a function `f = (λ (x) ...)` in terms of itself, we wrap this function in a function binding `f`, i.e., `(λ (f) (λ (x) ...))`, and then compute a fixed point for this function, which would be a function f that is closed over a binding for itself. A fixed-point combinator is known as a Y combinator and may be defined for the call-by-value λ calculus like so:

```
U ≡ (λ (a) (a a))
```

```
Y ≡ (U
     (λ (y)
       (λ (f)
         (f (λ (x)
              (((U y) f) x))))))
```

To see how this may be used, consider the example shown below, where we calculate the factorial of the number *5* using the Y combinator.

```
((Y
  (λ (f)
    (λ (x)
      (if (= x 0)
          1
          (* x (f (- x 1)))))))
 5) ;⟶120
```

The Y combinator may be used to simulate recursion by using the U combinator instead of having the ability to directly create self-referential definitions. U is used to duplicate the function at each call so that it may both be called and also passed down to be called yet again if needed. Try the code above in Scheme and try to work out how each part of the Y combinator functions. How might you compile a `letrec` form in Scheme to the plain λ calculus using Y?